Software Design Document (SDD)

System: Jima — CARAVAN Lounge Restaurant Management System
Document: SDD v1.1 · **Updated:** August 20, 2026

1. System Architecture

The system is a three-tier logical application and modular monolith. The registered Jima web artifact runs `artifacts/nextjs-app` (frontend). `artifacts/nestjs-server` is the authoritative restaurant backend. `artifacts/api-server` is a separate minimal artifact and must not be confused with the restaurant API.

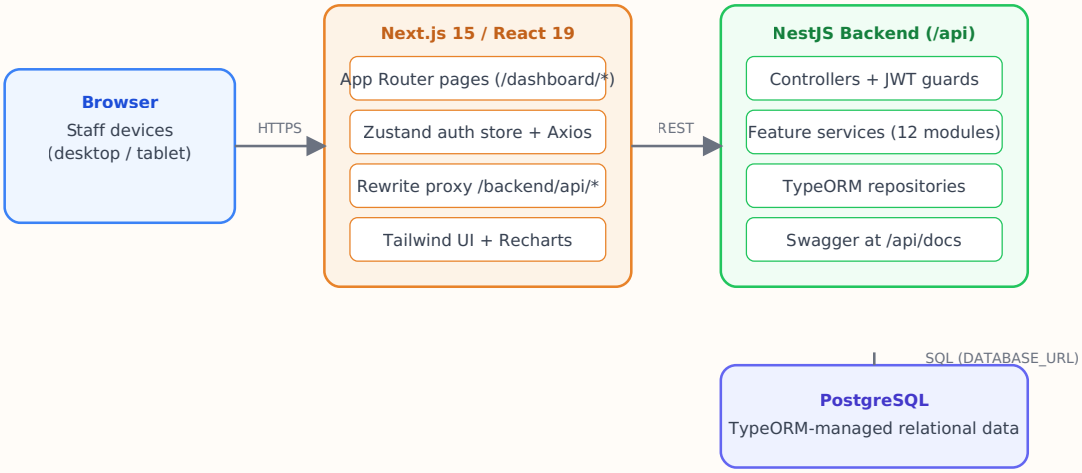

Figure 1 — High-level architecture. The browser talks only to the frontend origin; API calls are proxied to NestJS, which owns all database access.

2. Frontend Structure

Path	Responsibility

`src/app/login`	Login page (photo hero, credential form).
`src/app/dashboard/page.tsx`	Role-aware landing: renders ManagerDashboard for admin/owner/manager; redirects waiter/coordinator/cashier to Orders.
`src/app/dashboard/*`	Feature pages: orders (POS), order-board, kitchen, tables, menu, inventory, item-requests, requests, staff, branches, reports.
`src/components`	Shared UI: sidebar (role-filtered nav), manager-dashboard, stat cards, etc.
`src/lib/api.ts`	Browser API facade; attaches JWT and calls the same-origin `/backend/api` boundary.
`src/store/auth.ts`	Zustand store: current user, token, login/logout.

Data freshness is achieved with a uniform pattern: each operational page runs `fetchData()` on mount and every 10 s via `setInterval`, without toggling loading state (no flicker).

3. Backend Structure

Module	Endpoints (prefix /api)	Notes
Auth	POST /auth/login, GET /auth/me	bcrypt verify → JWT (Passport strategy)
Users	/users, /users/staff-list, /users/waiters, /users/chefs	Role management
Restaurants / Branches	/restaurants, /branches	Company hierarchy
Menus	/menus (categories + items)	Availability flags feed POS
Tables	/tables	Status transitions
Orders	/orders (+ /stats, /alerts, item edits, status)	Lifecycle enforcement
Kitchen	/kitchen (board, accept/preparing/ready)	Chef queues

Payments	/payments (+ report, shifts)	Marks orders paid, frees tables
Inventory	/inventory (items, low-stock, suppliers, purchase orders, requests, adjustments)	Store operations
Notifications	/notifications	Role/user targeted
Seed	POST /seed	Demo data (Ethiopian menu, staff)

4. Database Design (ER Overview)

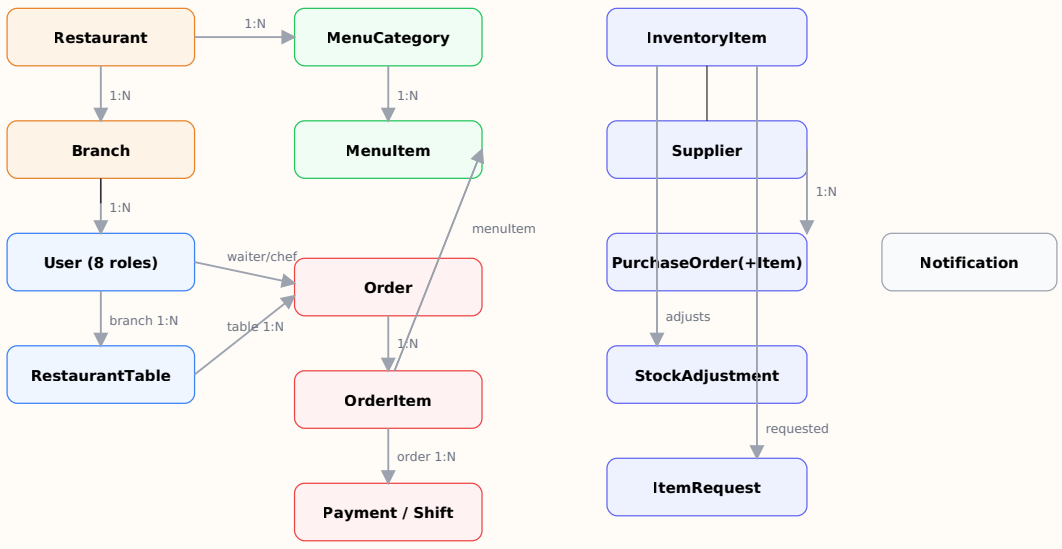

Figure 2 — Entity-relationship overview (key relations only). All entities carry auto-increment ids and timestamps where relevant.

Key columns per entity are listed in the SRS §2.3 environment and in the backend entity files (`artifacts/nestjs-server/src/**/*.entity.ts`), the authoritative source. Notable choices:

- `Order.status` enum drives the entire workflow; `OrderItem` cascades on order delete.
- `User.password` uses `select:false` so hashes never leak into queries by default.

- `ItemRequest` keeps requester / approver / issuer user references for auditability.

5. Order Lifecycle (Sequence)

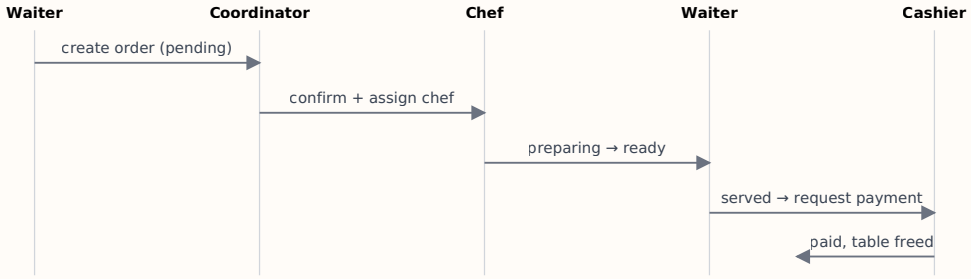

Figure 3 — Happy-path order sequence across roles.

6. Cross-Cutting Design Decisions

Concern	Decision
Auth	Stateless JWT (Passport). Token in the Zustand store; Axios interceptor adds the header.
API routing	Backend global prefix `/api`; frontend proxies `/backend/api/*` via Next.js rewrites so the browser stays same-origin.
Realtime	10-second polling everywhere (simple, proxy-friendly, no WebSocket infra).
Schema	TypeORM `synchronize:true` for development speed; switch to migrations for production (see Handoff doc).
Exports	Client-side generation: `xlsx` for Excel, `jspdf` + `jspdf-autotable` for PDF, loaded dynamically.
Styling	Tailwind CSS with shared utility classes (`.card`, `.input`, `.table-cell`) in `globals.css`; brand color #E8832A.
Profitability	Food cost estimated from stock deduction/waste movements × item unit cost (documented in User Manual).

7. Boundary and Readiness Notes

- The frontend owns presentation, navigation, forms, loading/error states, and API communication. It must never receive database credentials or own authoritative business permissions.
- The NestJS backend owns authentication, authorization, role/branch scope, validation, transactions, business rules, and all database access.
- PostgreSQL owns persistence and relational integrity. TypeORM entities are currently authoritative, but production migration evidence is not yet complete.
- The browser contract remains `/backend/api/*`; changing it requires coordinated frontend, rewrite, backend, deployment, and documentation updates.
- Current production blockers include schema synchronization, a fallback JWT secret, startup account mutation, wildcard CORS, incomplete DTO validation, and incomplete automated tests.

Detailed routing, environment, API, database, testing, and readiness rules are maintained in documents 07–12.